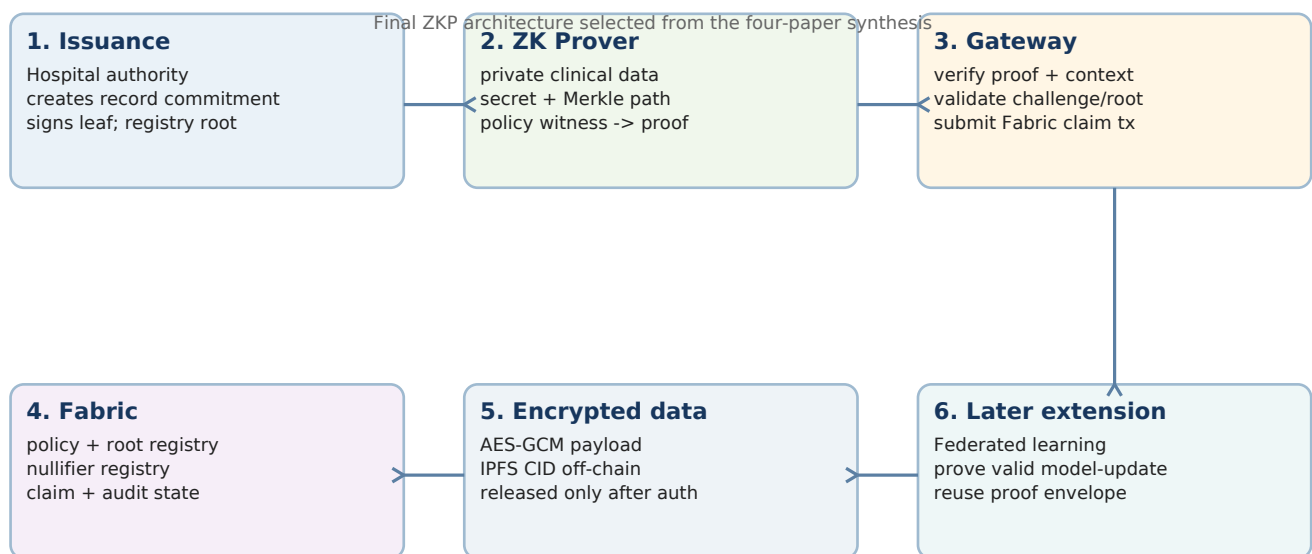


ZeroTrustBlock

Integrated ZKP Implementation Result and Engineering Blueprint

Consolidated from four healthcare/privacy ZKP papers and mapped onto the current
norn12/hyperledger repository



Document status: implementation target frozen for the ZKP phase. This document is the result of the literature synthesis and repository analysis; the codebase has not yet been modified by this document.

Scope: ZKP only. Federated learning is intentionally excluded from the implementation work described here. It will be introduced later using the proof/commitment interfaces designed in this phase.

Final decision: use the 2026 health-insurance eligibility paper as the primary cryptographic design; borrow dynamic policy binding/revocation ideas from ZK-EHR, Fabric integration and composable gadget lessons from Harpocrates, and structured healthcare/FHIR representation lessons from the 2025 healthcare data-sharing paper.

1. Executive Implementation Decision

The four papers do not point to a single system that should be copied. They provide complementary design components. The best implementation for ZeroTrustBlock is therefore a paper-guided hybrid with one coherent proof relation.

Decision	Final choice	Reason
Primary proof model	Composite health-claim eligibility proof	Closest match to the intended privacy problem; proves several healthcare/insurance predicates together rather than isolated demo checks.
Proof system	Groth16	Small proofs and fast verification; already fits the current project. Benchmark alternatives rather than changing systems prematurely.
In-circuit hash	Poseidon	ZK-friendly commitment and Merkle operations; avoids putting SHA-256 bit logic inside the circuit.
Record authentication	Authority signature inside the proof	Binds the hidden record to an authorized healthcare issuer.
Record existence	Merkle membership	Lets the prover establish registry membership without exposing the record.
Policy binding	Policy commitment + Fabric-registered policy state	Prevents the prover from choosing favorable policy parameters.
Replay protection	Record-bound nullifier	Prevents repeated use while avoiding direct patient identity disclosure.
Freshness/front-running	Gateway challenge + recipient/context binding	Adapts the paper's recipient-binding idea to Fabric rather than Ethereum.
Fabric role	State, policy, roots, nullifiers, claim/audit records	Fabric becomes the authoritative coordination and enforcement layer.
Off-chain data	Retain encrypted IPFS	ZKP verifies properties of hidden data; IPFS continues to hold encrypted payloads.
Current gnark	Upgrade before signature-circuit work	The repository is on gnark 0.9.1; this is below patched releases for known signature-malleability issues.
Circom	Optional reproduction/reference harness	The primary paper uses Circom/snarkjs; final repo integration should remain Go-native unless comparison shows a compelling reason to switch.

Upstream gnark currently lists v0.16.3 among its releases, while the published EdDSA/ECDSA signature-malleability advisory affects versions below 0.13.0 and lists v0.14.0+ as patched. The current repository pins gnark v0.9.1, so signature-circuit implementation must not begin on the current dependency. Exact version pinning should happen after the upgraded build and regression suite pass. [5, 6]

2. What We Take from All Four Papers

Paper	What it contributes	What we adopt	What we do not copy
Werapun & Werapun (2026), Cryptography	Full health-insurance eligibility relation: signed records, Merkle registry, composite predicates, nullifier, recipient binding, domain separation; extensive circuit/performance analysis.	Core relation, threat model, record-bound nullifier, policy/registry binding, hardening and benchmark methodology.	Ethereum/EVM contract mechanics and gas-centric results; we adapt them to Fabric.
Albalwy (2026), Systems - ZK-EHR	Privacy-preserving policy enforcement, Poseidon-based policy hashing, role/attribute proof, encrypted IPFS workflow, proposed expiry and replay protection.	Policy commitment idea, credential binding, expiry/freshness concepts, separation of prover/verifier/storage.	Its tiny 5-constraint role circuit is not sufficient as our principal research circuit; several security extensions are proposed rather than fully implemented.
Myeong & Sa-Ram (2025), JTIE	zk-SNARK healthcare-sharing protocol with structured FHIR data and interoperability-focused APIs; permissioned Ethereum implementation.	Healthcare data abstraction/FHIR-compatible boundary and interoperability mindset.	Ethereum-specific smart contract layer and exact circuit design.
Thazhath, Michalak & Hoang (2022), Harpocrates	Real Hyperledger Fabric implementation using ZKP gadgets, Poseidon Merkle trees, commitments, and explicit performance measurements.	Fabric-oriented integration lessons, composable gadget architecture, metrics: constraints, proving/verification delay, memory, bandwidth.	Bulletproofs as the primary backend; our system retains Groth16 unless experiments justify otherwise.

The 2026 insurance paper explicitly reports a composite relation containing record commitment, authority signature, Merkle membership, diagnosis membership, 64-bit range constraints, a record-bound nullifier, and recipient binding. Its canonical hardened depth-20 circuit is 10,050 constraints; proof size is 256 bytes; verification is a fixed pairing check whose cost depends on public-input count. [1]

ZK-EHR provides a useful complementary lesson: policy values can be represented through a hash/commitment rather than hard-coded constants, while credential expiry and request-specific nullifiers can be layered around Groth16. The authors explicitly describe these as extensions and note that their full implementation and evaluation remain future work. [2]

Harpocrates is the strongest reference for the Fabric side because it was actually implemented with Hyperledger Fabric and measured circuit complexity, proving/verification delay, memory and bandwidth. Its operations ranged from 2,389 constraints for Store to about 86-87k constraints for membership-heavy operations. [4]

The 2025 JTIE protocol is useful mainly for healthcare interoperability: its implementation uses Python zk-circuits, Solidity contracts, REST APIs and HL7 FHIR, and it reports verification under 100 ms in its setting. We therefore treat FHIR as an interface/modeling option, not as the cryptographic core. [3]

3. Current Repository Baseline and Required Migration

The current repository already has a good infrastructure base, but its ZKP layer is still a prototype. The current Gateway generates an age-range Groth16 proof during write/read operations and passes only a SHA-256 proof hash to chaincode. The chaincode checks that the proof hash is non-empty; it does not perform Groth16 verification itself. The current ZKP package contains AgeRangeCircuit and DiagnosisCategoryCircuit.

Current area	Current state	Required change
zkp/health_circuits.go	AgeRange + numeric diagnosis range	Replace with composite HealthClaimCore/Hardened relation; keep legacy implementation only for baseline experiments.

Current area	Current state	Required change
gateway/gateway.go	Proof generated automatically in WriteHealthRecord and ReadHealthRecord	Separate issuance from claim/access proof. Add dedicated proof service and proof envelope.
chaincode/main.go	ZKPProofHash is stored; access policy treats non-empty hash as evidence	Add policy, root/version, challenge, nullifier and claim state. Do not treat non-empty hash as cryptographic verification.
Fabric	2 orgs, 4 peers, 3 Raft orderers, 1 healthchannel	Retain topology initially; add authoritative registry/policy state.
IPFS	AES-GCM encrypted payload + CID	Retain. ZKP proves claims about hidden data; IPFS remains off-chain storage.
benchmarks	Age-only ZKP and Fabric/E2E measurements	Add circuit-variant, Merkle-depth, policy-size, security and Gateway overhead experiments.
identity	Fabric MSP/role identity	Keep for organization/role authorization; do not expose raw patient identity as a ZKP public input.

Important: the current repository is useful as the baseline. It should not constrain the final design. We preserve working infrastructure while replacing security-critical ZKP semantics.

4. Final ZKP Relation

The principal circuit should prove a single meaningful statement: “There exists a valid authority-issued healthcare record, registered in the current accepted registry, whose hidden attributes satisfy the registered policy and this specific request, without revealing those attributes.”

Public statement x:

```
x = (
  registryRoot,
  rootVersion,
  policyCommitment,
  claimAmount,
  authorityPublicKey,
  nullifier,
  challengeCommitment,
  domainSeparator,
  recipientCommitment
)
```

Private witness w:

```
w = (
  diagnosis,
  labValue,
  policyId,
  coverageCeiling,
  patientSecret,
  authoritySignature,
  merkleSiblings[],
  merkleDirections[],
  recordIndex,
  optional treatment/date witnesses
)
```

The circuit enforces the following relation:

1. $k = \text{Poseidon}(\text{patientSecret})$
2. $\text{leaf} = \text{Poseidon}(\text{diagnosis}, \text{labValue}, \text{policyId}, \text{coverageCeiling}, k)$
3. $\text{authoritySignature}$ verifies over the leaf
4. every Merkle direction bit is Boolean
5. $\text{Merkle path}(\text{leaf}, \text{siblings}, \text{directions}) == \text{registryRoot}$
6. diagnosis is in the policy-covered set
7. $\text{labMin} \leq \text{labValue} \leq \text{labMax}$, with explicit 64-bit range discipline
8. $\text{claimAmount} \leq \text{coverageCeiling}$
9. policyCommitment recomputes to the registered policy
10. $\text{nullifier} = \text{Poseidon}(\text{patientSecret}, \text{recordIndex})$
11. $\text{challenge/recipient/domain}$ values are bound to the proof statement

The insurance paper gives the exact core relation structure, including the key warning that the nullifier must be derived from a record-bound index/path rather than a prover-chosen claim ID, and that Merkle direction bits must be constrained to $\{0,1\}$. [1]

5. Issuance and Commitment Workflow

Do not generate a ZKP automatically when writing the record. Replace the current write-time proof with a registry/issuance operation.

PATIENT	HOSPITAL / AUTHORITY	FABRIC
-- generate secret s ----->		
	compute/sign record leaf	
	-- register root/metadata ----->	
<-- signed record + witness -----		
+---- retain secret and private data		

Blinded issuance: the patient retains the secret used for the nullifier/commitment. The issuer should not learn this secret. This preserves the unlinkability goal described in the primary paper. The paper also notes an operational trade-off: losing the secret makes the record unclaimable; a backed-up seed-derived per-record secret can reduce accidental loss while keeping each per-record secret individually unpredictable. [1]

6. Merkle Registry Design

The registry is not just a data structure; it is the bridge between the private record and Fabric's authoritative state.

```
Record leaf
|
+--> Poseidon(diagnosis, labValue, policyId, coverageCeiling, Poseidon(secret))
|
+--> authority signature over leaf
|
+--> append leaf to Merkle registry
|
+--> root_v1
+--> root_v2
+--> root_v3
...
```

Fabric should maintain a bounded recent-root window. The primary paper uses a 32-root ring buffer to preserve liveness when issuance advances the root while proofs are still in flight; for ZeroTrustBlock, the window should be experimentally chosen rather than copied as a universal constant. [1]

Required Fabric root state:

```
MerkleRoot {
  version
  root
  createdAt
```

```

    active / accepted
  }

  RecentRoots:
    version -> root

```

7. Policy Representation

The current repository accepts an arbitrary JSON `accessPolicy` string from the Gateway. That is not sufficient for a cryptographic policy claim. The authoritative policy must live in Fabric and be committed into the ZKP public statement.

```

Policy {
  policyID
  version
  diagnosisRoot / diagnosisSetCommitment
  treatmentRoot / treatmentSetCommitment
  labMin
  labMax
  maxClaimAmount or hidden coverage ceiling rule
  validityStart
  validityEnd
  requiredRole / organization policy
}

policyCommitment = Poseidon(policyID, version, ... policy fields ...)

```

ZK-EHR's design supports this direction: it proposes replacing hard-coded equality checks with Poseidon-based attribute hashing against an on-chain policy hash. It also proposes credential binding and expiry checks. Those ideas fit naturally into our policy-commitment model. [2]

8. Predicate Design

Predicate	Implementation	Privacy behavior
Diagnosis	Membership in policy-covered diagnosis set; start with fixed small set for the baseline, then use Merkle-backed set for scale.	Exact diagnosis remains private.
Treatment	Same pattern as diagnosis, optionally added after diagnosis membership passes.	Exact treatment remains private.
Lab value	Explicit fixed-width range proof (64-bit discipline) against policy bounds.	Exact value remains private.
Claim amount	<code>claimAmount <= coverageCeiling</code> ; begin with public claim amount and hidden ceiling, then evaluate fully hidden amount as an extension.	Policy ceiling can remain private.
Age	Retain only as an optional predicate once the composite circuit is stable.	Age is private unless explicitly needed by the policy.
Waiting period	Date relation with private dates + public policy window/version; add after core relation passes.	Exact dates can remain private.

The primary paper uses true set membership for diagnosis, explicit 64-bit comparator discipline for ranges, and a hidden coverage ceiling. This is preferable to the current `DiagnosisCategoryCircuit``, which only accepts a numeric interval and therefore does not represent true set membership. [1]

9. Nullifier, Freshness, and Context Binding

```
recordIndex = recovered from the Boolean-constrained Merkle path
nullifier    = Poseidon(patientSecret, recordIndex)

challengeCommitment = Poseidon(challenge, requesterContext)

domainSeparator = Poseidon(
    "ZeroTrustBlock",
    channelID,
    chaincodeID,
    circuitVersion
)
```

Why recordIndex, not claimID? A prover-chosen claim ID can be changed and would allow the same record to produce fresh nullifiers. The primary paper explicitly identifies this as a soundness failure of the anti-double-claim mechanism. [1]

Why a challenge? It replaces Ethereum-style recipient/front-running binding with a Fabric-compatible session/request mechanism. The challenge is generated for a specific request, stored or authenticated by Fabric, and included in the public proof statement.

Why domain separation? It prevents a proof generated for one ZeroTrustBlock deployment/context from being silently reused in another context. The primary paper demonstrates this as an explicit hardening check. [1]

10. Authority Signature and Identity Model

Use two identity planes:

```
Fabric identity plane
  X.509 / MSP / role attribute
  -> organization-level authorization

ZK authority plane
  ZK signing key
  -> proves the committed record was authorized by the issuer
```

Do not put the patient's raw Fabric identity in the public witness. Use a commitment/secret relationship. The issuer's ZK public key can be registered in Fabric and referenced by policy/network state. The signature verification circuit should use a patched cryptographic stack, not the repository's current gnark 0.9.1. The gnark advisory specifically describes EdDSA/ECDSA signature malleability caused by missing scalar checks in affected versions. [cite]turn579945search3

11. Two-Circuit Strategy: Core vs Hardened

Use two explicit circuit versions so the thesis can show the cost of security hardening rather than presenting one opaque final number.

Circuit	Contains	Purpose
HealthClaimCore	commitment + signature + Merkle membership + diagnosis membership + range + claim ceiling + record-bound nullifier	Functional cryptographic baseline.
HealthClaimHardened	Core + Booleanity + policy commitment + domain separation + challenge/recipient binding + explicit context checks	Final security target.

This mirrors the primary paper's most useful methodological feature: separate the functional relation from the hardening changes and re-measure the final relation instead of silently applying old measurements to a new circuit. The paper explicitly distinguishes its primary wall-clock campaign from its later hardened canonical relation. [1]

12. Exact Repository Changes

Path	Action	Implementation result
zkp/legacy/	Create/move old circuits	Preserve AgeRange and DiagnosisCategory only for baseline comparison.
zkp/health_claim_circuit.go	Create	Composite circuit definition / public-private signal model.
zkp/commitment.go	Create	Poseidon record/policy/nullifier/context commitments.
zkp/merkle.go	Create	Merkle path generation/verification and root-version handling.
zkp/signature.go	Create	Authority-key/signature helpers using upgraded cryptographic dependency.
zkp/service.go	Refactor/create	Setup, witness building, Prove, Verify, serialization, circuit-version selection.
zkp/health_claim_test.go	Create	Positive/negative relation tests and adversarial context tests.
zkp/benchmarks/	Create	Constraint/prover/verifier/memory/proof-size benchmarks.
gateway/zkp_client.go	Create	Dedicated ZKP interface; no automatic age proof in read/write.
gateway/proof_models.go	Create	ProofEnvelope, public input schema, circuit/version metadata.
gateway/gateway.go	Refactor	IssueRecord and SubmitClaim/VerifyProof flow; retain IPFS integration separately.
chaincode/models.go	Create/refactor	Policy, MerkleRoot, Claim, Nullifier, Challenge, audit structures.
chaincode/main.go	Refactor	Registry and claim transactions; eliminate non-empty-hash-as-verification semantics.
chaincode/policy.go	Create	Policy registration/update and policy commitment state.
chaincode/registry.go	Create	Merkle root window + accepted versions.
chaincode/claims.go	Create	Claim submission, nullifier check, challenge binding, result/audit.
experiments/	Extend	Core vs hardened, Merkle scaling, policy scaling, security matrix, Gateway overhead.

13. Gateway Workflow

The current Gateway is the orchestration point. Its role should be reduced to a clear protocol adapter, not the owner of the privacy boundary.

A. Registration / issuance

Client -> Gateway -> Hospital authority -> Fabric registry

B. Challenge

Requester -> Gateway -> Fabric

Fabric -> challenge / request context

C. Proof generation

Prover (final target: client-side)

private record + secret + Merkle witness + policy witness

-> HealthClaimHardened proof

D. Verification

Gateway receives:

proof + public inputs + challenge + circuit version

-> verify

-> fetch authoritative Fabric state

-> compare policy/root/version/context

E. Claim submission

Gateway -> Fabric SubmitClaimProof

Fabric:

check challenge


```

check root window
check nullifier unused
check policy commitment
enforce verifier result / verification path
record claim + nullifier + audit

```

Privacy boundary: the final architecture should allow the prover to keep the private witness outside the Gateway. The Gateway receives only proof material and public statement fields. During development, the Gateway may host the prover for easier testing, but this should be explicitly labeled “development mode,” not the final privacy guarantee.

14. Fabric Integration Stages

Stage	Fabric authority	What changes
F1	Metadata + policy + root state	Add Policy, MerkleRoot, Challenge, Nullifier and Claim records.
F2	Gateway verifies Groth16	Fabric records proof envelope metadata and verifies all state bindings.
F3	Authoritative verifier	Replace “non-empty proof hash” logic with an actually trusted proof-verification path.
F4	Audit/privacy hardening	Do not leak raw patient ID, clinical values or unnecessary access metadata.

The primary paper’s on-chain sequence is: bind public inputs to registered state, reject a used nullifier, verify the proof, then record the nullifier and approve the claim. This ordering should be mirrored in the Fabric transaction semantics as closely as the platform allows. [1]

Harpocrates demonstrates that Fabric can be used as a distributed audit/coordination layer while the ZKP cryptographic operations are handled through dedicated circuits/gadgets. [4]

15. IPFS Role - Keep, but Narrow It

```

Encrypted healthcare payload
|
+--> AES-GCM encryption
+--> IPFS CID
|
+--> Fabric stores CID + data hash + record metadata

```

ZKP proves:
“the hidden committed record satisfies the policy/request”

IPFS provides:
“the encrypted payload can be retrieved after authorization”

The 2026 ZK-EHR paper also separates proof-based authorization from encrypted IPFS retrieval. It additionally notes that revocation cannot retroactively retract a file once a party already possesses the decryption capability, so key-release/key-rotation policy is a separate security layer. [2]

16. Security Test Matrix

Test	Mutation	Expected result
Valid proof	All fields valid	Accepted
Diagnosis exclusion	Diagnosis not in covered set	Proof generation/verification fails
Lab lower bound	labValue < min	Fails
Lab upper bound	labValue > max	Fails
Claim ceiling	claimAmount > coverageCeiling	Fails
Bad authority signature	Change signature/leaf	Fails
Wrong Merkle sibling	Modify path element	Fails
Non-Boolean path selector	Set direction value outside {0,1}	Fails
Wrong policy commitment	Verify proof under another policy commitment	Fails
Wrong registry root	Use root from another registry/version outside window	Fails
Wrong circuit version	Use proof against another verification key/version	Fails
Wrong domain	Change deployment/domain separator	Fails
Wrong challenge	Replay proof with new/other challenge	Fails
Wrong recipient context	Use proof for another requester context	Fails
Nullifier replay	Submit same proof/record twice	Second claim rejected
Public-input tampering	Modify any public signal	Verification fails
Under-constrainedness	Static analysis + adversarial witness search	No exploitable unconstrained signal

The Booleanity check is especially important. The primary paper reports that `MultiMux1` interpolates over its selector; without explicit $b(1-b)=0$, a malicious prover could choose non-Boolean path selectors and manipulate the reconstructed root. [1]

17. Performance Evaluation Plan

Do not reuse paper timings as our result. Use them as reference points. Harpocrates and the two 2026 papers demonstrate that circuit complexity, proving/verification time, memory, proof size and storage/network overhead are measurable and should be reported together. [1, 2, 4]

Dimension	Metrics to report
Circuit	R1CS constraints, multiplication gates if available, private/public signals, proving key size, verification key size.
Setup	Setup time, artifact size, ceremony/setup assumptions.
Prover	Witness generation, proving time mean/P50/P95/P99, peak RSS, CPU utilization.
Verifier	Verification time mean/P50/P95/P99, memory, throughput.
Proof	Serialized proof bytes, public input bytes, envelope size.
Merkle scaling	Depth 8/12/16/20/24; constraints, proving, verification, memory.
Policy scaling	8/16/32/64 covered entries; compare explicit membership vs Merkle-backed membership.
Hardening cost	Core vs Hardened delta for constraints/time/memory/proof envelope.
Gateway	Request parsing, witness build, proof generation, proof verification, Fabric submit, end-to-end latency.
Fabric	Accepted claims/s, latency, failure rate, nullifier storage growth, root-window lookup overhead.

Dimension	Metrics to report
Bandwidth	Proof + public-input + transaction payload size.

The 2026 insurance paper reports depth-16/20/24 performance and shows that Merkle depth drives circuit size while Groth16 proof size stays constant for the fixed relation/public-input layout. It also reports that a policy predicate set adds relatively little circuit cost compared with signature and membership machinery. These are useful hypotheses to reproduce, not numbers to claim for ZeroTrustBlock. [1]

18. Benchmark Experiments to Run

Experiment	Configuration	Primary question
E1 Baseline legacy	Current AgeRangeCircuit	What does the original system cost?
E2 Core circuit	HealthClaimCore	What does the paper-guided cryptographic relation cost?
E3 Hardened circuit	HealthClaimHardened	What does security hardening cost?
E4 Merkle scaling	Depth 8/12/16/20/24	How does authenticated registry size affect proving?
E5 Policy scaling	8/16/32/64 entries	When does Merkle membership become more attractive than explicit set membership?
E6 Gateway decomposition	Prove / Verify / Fabric separately	Where is the dominant end-to-end overhead?
E7 Replay/security	All adversarial cases	Does every stated security property fail closed?
E8 Fabric baseline	Claim transaction without proof verification	What is Fabric-only cost?
E9 Full path	Real proof + Gateway + Fabric	What is privacy-enabled end-to-end cost?

19. Expected Implementation Result

At the end of the ZKP phase, the repository should be able to execute the following complete scenario.

1. Hospital registers an authority-signed committed health record.
2. Patient retains a private record secret.
3. Fabric stores the policy commitment and current/recent Merkle roots.
4. Requester obtains a fresh challenge.
5. Prover constructs a private witness.
6. HealthClaimHardened generates a Groth16 proof.
7. Gateway verifies proof + public statement.
8. Fabric checks policy/root/challenge/nullifier state.
9. First valid claim/access is accepted and logged.
10. Reuse of the same record produces the same record-bound nullifier and is rejected.
11. Changing diagnosis, lab value, policy, root, domain or challenge invalidates verification.
12. The actual clinical document remains encrypted/off-chain.

The final proof envelope should contain metadata such as:

```
ProofEnvelope {
  circuitID
  circuitVersion
  proof
  publicInputs
  proofHash
  policyCommitment
  rootVersion
  nullifier
  challengeCommitment
  domainSeparator
}
```

20. Implementation Sequence - Do It in This Order

Phase	Work	Done when
Z0	Freeze current repo as baseline; record versions and current benchmark outputs.	Baseline reproducibly documented.
Z1	Upgrade cryptographic dependency; create new ZKP module boundary; retain legacy circuits.	Build/tests pass on patched dependency.
Z2	Implement commitments and data model.	Leaf/policy/nullifier/context vectors match reference calculations.
Z3	Implement Merkle registry and Boolean path constraints.	Valid and malicious path tests pass.
Z4	Implement authority signature verification.	Valid signature passes; malformed/malleated/incorrect signature fails.
Z5	Implement policy predicates.	Diagnosis membership, range and claim ceiling negative tests pass.
Z6	Compose HealthClaimCore.	Core proof passes end-to-end locally.
Z7	Add policy commitment, domain separation, challenge/recipient binding.	Hardened relation passes all context tests.
Z8	Security analysis.	Static analysis + adversarial test matrix complete.
Z9	Performance evaluation.	E1-E5 complete with reproducible environment data.
Z10	Gateway integration.	Gateway verifies proof envelope without requiring raw clinical data in final mode.
Z11	Fabric integration.	Policy/root/nullifier/challenge/claim workflow is authoritative; no proof-hash-only authorization.
Z12	End-to-end evaluation.	E6-E9 complete and thesis-ready tables/plots generated.

21. What We Explicitly Do NOT Implement in the First ZKP Milestone

- No full EHR-in-circuit. Keep the record off-chain and prove only the properties needed for the claim.
- No full neural-network proof. Federated learning is later; this ZKP layer must remain domain- and proof-envelope-oriented.
- No automatic age proof on every read/write. Eligibility is a claim/request workflow, not a side effect of record creation.
- No arbitrary client-supplied access policy. The authoritative policy is registered and committed.
- No proof-hash-as-proof semantics. A hash is an identifier/integrity artifact, not proof verification.
- No immediate migration to Bulletproofs/PLONK. Benchmark them as alternatives; do not add a second proving system before the primary relation works.

22. Future Extensions After the ZKP Baseline

Extension	Why it matters	Priority
Dynamic treatment membership	Generalizes diagnosis membership to procedures/treatments.	High
Expiry + revocation	Turns proposed ZK-EHR operational extensions into measured implementation.	High
Private claim amount	Move from public claim amount/hidden ceiling to commitment + range proof.	Medium
Multi-authority	Hospital + insurer + lab / regulator attestations.	Medium
PLONK/transparent comparison	Quantify universal/transparent setup tradeoffs.	Medium
BLS12-381 experiment	Security/curve comparison if hardware/toolchain supports it.	Low/optional
Recursive/aggregated proofs	Useful when many proofs must be combined.	Later
Federated learning	Reuse ProofEnvelope + commitment/context mechanisms to prove bounded/valid model updates.	Later - separate project phase

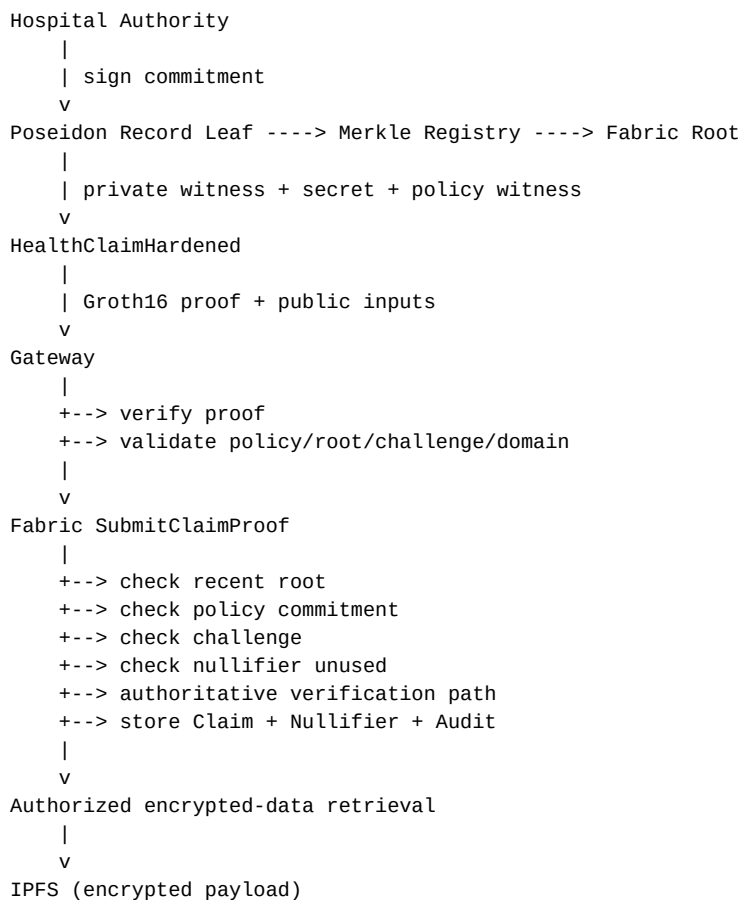
ZK-EHR explicitly lists revocation, replay prevention, richer policy logic, credential binding and operational key management as extensions. The primary 2026 insurance paper identifies dynamic Merkle updates, domain separation, under-constrainedness analysis, PLONK/alternative proving systems, recursive proofs, multi-authority support and post-quantum systems as future directions. These become a prioritized backlog rather than requirements for the first implementation. [1, 2]

23. ZKP Phase Acceptance Criteria

Category	Pass condition
Privacy	Clinical diagnosis/lab/treatment values are absent from proof public inputs and blockchain claim state unless explicitly required.
Soundness	No adversarial witness can make an invalid policy claim verify in the test suite.
Context binding	Wrong policy/root/domain/challenge/recipient fails.
Replay resistance	Same record cannot be claimed twice because the record-bound nullifier is reused and rejected.
Registry integrity	Merkle root/path and authority signature are both enforced.
Policy authority	Prover cannot choose its own favorable policy parameters.
Fabric authority	Claim state is tied to authoritative on-chain policy/root/nullifier state.
Performance	Core and hardened costs are independently measured; no paper number is relabeled as a ZeroTrustBlock result.
Reproducibility	Setup scripts, tool versions, dataset vectors, benchmark commands and output files are versioned.

24. Final Architecture in One View

ZeroTrustBlock ZKP



Later: the same ProofEnvelope/context-binding machinery can carry privacy proofs for federated-learning model updates.

Bottom line: the ZKP phase should produce a real privacy-preserving claim/authorization primitive, not a collection of demo circuits. The primary 2026 insurance paper supplies the cryptographic backbone; ZK-EHR supplies policy/freshness ideas; Harpocrates supplies the Fabric/composable-gadget perspective; JTIE supplies the healthcare interoperability boundary.

25. Sources Used for the Four-Paper Synthesis

Werapun, W.; Werapun, W. (2026). Privacy-Preserving Health Insurance Eligibility Verification on Blockchain Using zk-SNARKs. *Cryptography* 10(5), 66. DOI: 10.3390/cryptography10050066.
<https://www.mdpi.com/2410-387X/10/5/66>

Albalwy, F. (2026). Zero-Knowledge-Based Policy Enforcement for Privacy-Preserving Cross-Institutional Health Data Sharing on Blockchain. *Systems* 14(4), 385. DOI: 10.3390/systems14040385.
<https://www.mdpi.com/2079-8954/14/4/385>

Myeong, G. E.; Sa-Ram, K. (2025). Blockchain Based Zero Knowledge Proof Protocol For Privacy Preserving Healthcare Data Sharing. *Journal of Technology Informatics and Engineering* 4(1), 171-189. DOI: 10.51903/jtie.v4i1.296.
<https://jtie.stekom.ac.id/index.php/jtie/article/view/296>

Thazhath, M. B.; Michalak, J.; Hoang, T. (2022). Harpocrates: Privacy-Preserving and Immutable Audit Log for Sensitive Data Operations. *IEEE TPS*.
<https://arxiv.org/abs/2211.04741>

Harpocrates thesis artifact / performance appendix.
https://vtechworks.lib.vt.edu/bitstream/10919/110745/1/Thazhath_MB_T_2022.pdf

Consensys gnark releases.
<https://github.com/Consensys-Incorporated/gnark/releases>

Consensys gnark security advisory: signature malleability in EdDSA/ECDSA.
<https://github.com/Consensys-Incorporated/gnark/security/advisories/GHSA-95v9-hv42-pwrj>

ZeroTrustBlock current repository baseline.
<https://github.com/norn12/hyperledger>

Source note: numerical performance values in this document are cited as paper-reported reference values, not ZeroTrustBlock measurements. ZeroTrustBlock measurements must be produced by the benchmark suite after implementation. The repository state described here is the state inspected for this design decision.